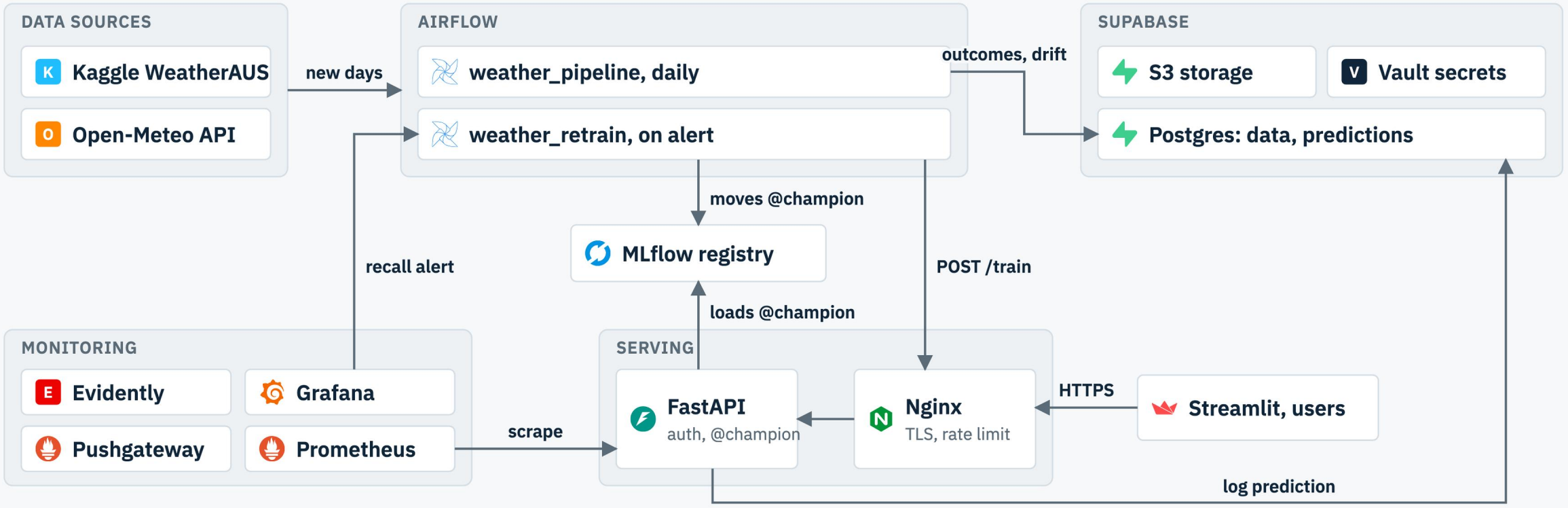
Will it rain tomorrow?

Next-day rain for 49 Australian weather stations,
trained, versioned and served as an MLOps pipeline.

Gabriel **Jonathan** **Thomas** **Ziad**

DataScientest MLOps defense, 15 October 2026

Overall software architecture



Closed loop: from prediction to new champion

Prediction logged → Real outcome arrives → Drift and recall check → Grafana alert → Airflow retrains → Gate moves @champion

Docker Compose: 11 services — 8 built from our 7 Dockerfiles (api and baseline share one), 3 public images. Airflow 2.8.1 runs as a separate stack.

Class imbalance: why accuracy is the wrong metric

77.1%

test accuracy of the majority-class baseline (always no)



- Target: RainTomorrow, binary. One XGBoost model for 49 stations.
- Promotion metric: validation ROC-AUC.
- Guardrail: validation recall of at least 0.75.

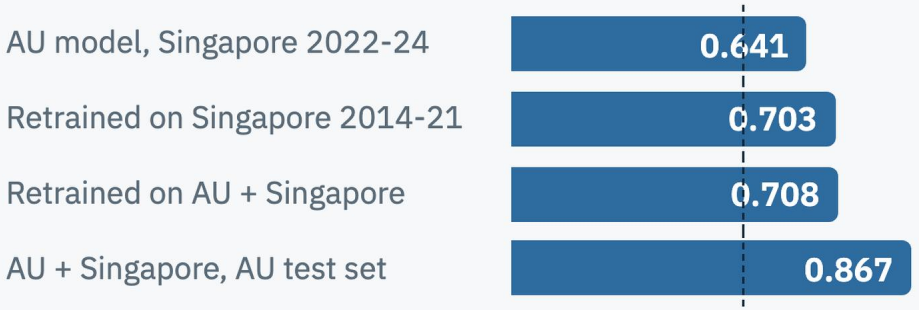
Test rows	**24,808**
Positive class (rain)	**5,689 (22.9%)**
Baseline recall	**0.000**
Candidate run: recall / precision	**0.762 / 0.543**
Candidate run: ROC-AUC	**0.866**

Distribution shift: Australia to Singapore

0.87 to 0.70

ROC-AUC, Australian test set vs all Singapore days

- Label shift: rain tomorrow on 82.6% of Singapore days, 21.9% in Australia.
- Covariate shift: Evidently (PSI) flags all 5 monitored features.
- Retraining recovers only part of the loss.



dashed line: 0.5, a random guess

Single ingress: the Nginx gateway

Every external request



- Only Nginx is public. All other tools bind to localhost.
- Excess requests get 429 before they reach Python.
- 6 live tests in CI: redirect, TLS, HSTS, health, 401, 429.

Public ports	**80 (301), 443**
API limit	**10 r/s per IP, burst 20**
/train limit	**1 r/s per IP, burst 3**
Body cap, timeouts	**2 MB, 15 s**
/metrics from outside	**404**

Data lineage: two sources, one schema, one hash

From source to training run



- Kaggle WeatherAUS: 145,460 daily station rows, 2007 to June 2017.
- Open-Meteo: same 23 columns.
- Split by date

DVC snapshot, 2 Oct	**185,150 rows, 49 stations**
Train, 5 years	**Mar 2014 to Mar 2019**
Validation, 90 days	**Mar to Jun 2019**
Test, 90 days	**Jun to Sep 2019**
Open-Meteo share: train, val, test	**35%, 100%, 100%**

Experiment tracking: every model and every day

Each training run, from make train, POST /train or Airflow

PARAMETERS

How was it trained?

- XGBoost settings: trees, depth, learning rate
- Training rows and class weight

METRICS

How good is it?

- Champion ROC-AUC 0.866
- Recall 0.776
- Precision 0.512

TAGS

Which data and code?

- `dataset_sha256`: the DVC snapshot
- `git_commit`: the code

ARTIFACTS

Can we reload it?

- The model itself
- Dataset manifest
- `release.json` ties them together

Each evening, the Evidently job on the latest predictions

DRIFT

Did inputs change?

- PSI per feature against the training data
- Share of drifted features

REAL PERFORMANCE

Was it right?

- Recall on predictions whose outcome is known
- Retrain request below 0.75

TAGS

Which model?

- Version holding `@champion`
- Hash of its training data

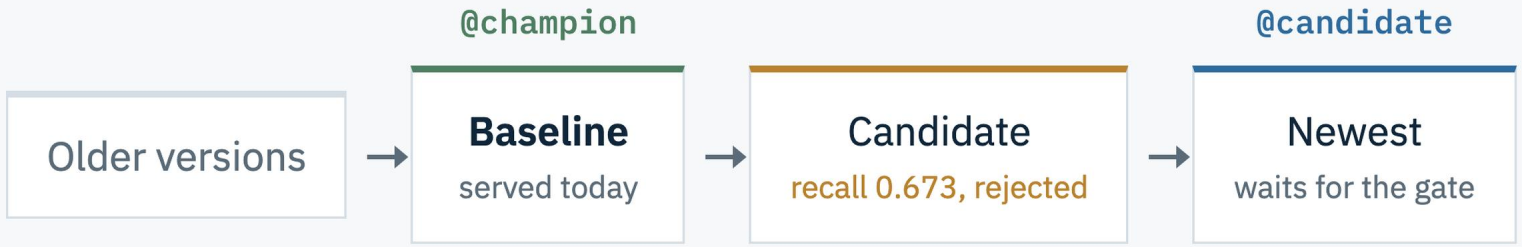
ARTIFACTS

Can we read it?

- Full Evidently HTML report, on S3
- `summary.json`

Model registry: only the gate moves @champion

Versions of one registered model, oldest to newest



The gate moves @champion only if all three hold

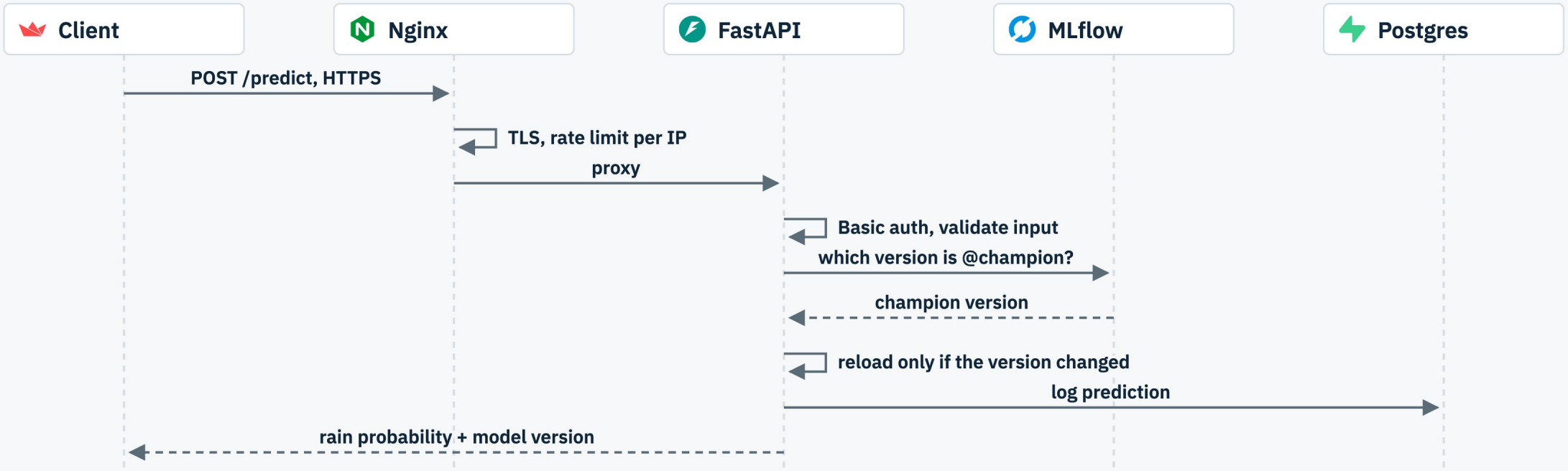
1. The release is complete: clean git commit, version matches its run, all files in MLflow.
2. Validation recall is at least 0.75.
3. Validation ROC-AUC beats the champion.

FastAPI asks for a name, not a file

`models:/weather-rainfall-classifier@champion`

It checks the alias on every prediction and reloads only when the version changes. Rollback: point the alias back.

Serving the champion

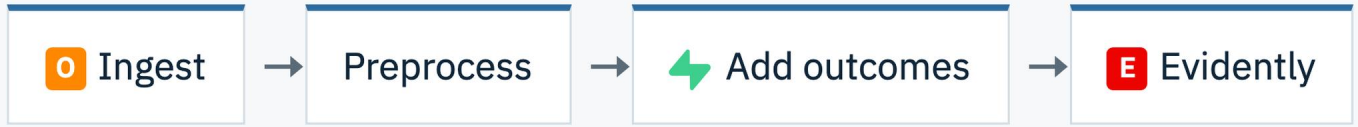


- Five public endpoints, plus /metrics for Prometheus only.
- Out-of-range values and unknown stations are rejected.
- Every answer names the model that produced it.

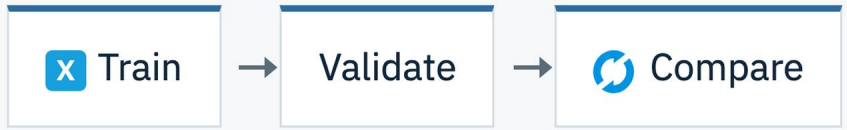
Endpoints	**5 + /metrics**
Rainfall bounds	**0 to 500 mm**
Stations accepted	**49**

Two DAGs: watch daily, retrain on demand

weather_pipeline, daily at 18:00 Berlin



weather_retrain, on Grafana alert



- Each task runs one of our Docker images.
- We check the model every day, but train only when it got worse.
- Training goes through the gateway, like any client.

DAGs	**2**
Schedule	**daily, 18:00 Berlin**
Retrain trigger	**Grafana webhook, max 1/day**
Missed days	**caught up automatically**

Continuous integration and delivery

CI, on pushes and pull requests to master

Lint → Unit tests → Build images → Live gateway tests

Release, on master

Build → Push to ghcr.io

CI jobs	**2**
Tests	**229 (6 live gateway)**
Images built	**7**
Images on ghcr.io	**3**

Monitoring with Prometheus and Grafana

Live, every 15 s



Daily, once the real weather is known



- The API reports traffic, errors, latency and every prediction.
- Evidently checks drift and the real recall once a day.
- Retraining starts only when the real recall drops, at most once a day.

Scrape interval	**15 s**
Dashboards	**2**
Alerts	**7 (3 API, 4 model)**
Max retraining	**1/day**

Live demo



Future work

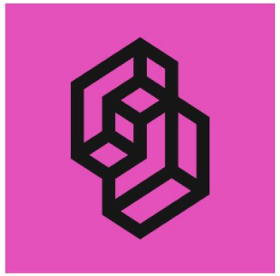
01



Scale on Kubernetes

Run resilient API replicas and increase capacity as live traffic grows.

02



Jev and transformers

Compare Jev and transformer models with the current XGBoost baseline.

03

Canary release

Route ten percent of live traffic to the candidate. Promote it only when real recall holds.

04

Shadow evaluation and rollback

Score candidates beside the champion. Monitor real recall and restore the previous champion when it drops.

Questions



Thank you